

Bonus task 1

PYTHON

Possibility to earn five bonus points

Deadline: 13 September 2026

The aim of this bonus task is to practise the basic use of Python and NumPy and to connect simple matrix operations to a simple finite element problem. In particular, the task illustrates why an element stiffness matrix is singular before sufficient boundary conditions are applied, and how boundary conditions make the reduced stiffness matrix solvable.

You can use the book *Programming for Computations – Python* as supporting learning material.

Python environment: Spyder

For this course, **Spyder** is recommended as the Python development environment. Spyder is particularly suitable for scientific and engineering computations, and its user interface is quite similar to MATLAB.

A typical Spyder window contains:

- an **Editor** for writing Python programs,
- an **IPython Console** for executing commands interactively,
- a **Variable Explorer** for inspecting variables, vectors and matrices, and
- tools for displaying plots and numerical results.

Thus, if you have previously used MATLAB, the basic workflow should feel quite familiar. You can test individual commands in the IPython Console and write longer programs in the Editor.

Try the IPython Console first

After starting Spyder, locate the **IPython Console**. You can type Python commands directly into the console.

For example, try

```
2 + 3
```

which should return

```
5
```

Then try

```
a = 4  
b = 6  
a * b
```

which should return

```
24
```

The variables `a` and `b` should now also be visible in Spyder's **Variable Explorer**.

You can also try a simple NumPy calculation:

```
import numpy as np

A = np.array([[2.0, 1.0],
              [1.0, 3.0]])

b = np.array([1.0, 2.0])

x = np.linalg.solve(A, b)

print(x)
```

Here, Python solves the linear system

$$x = A^{-1}b. \quad (1)$$

Later in finite element analysis, you will solve displacements of exactly the same general form,

$$u = K^{-1}f. \quad (2)$$

Writing and running a Python file in Spyder

A Python program is simply a text file with the extension `.py`. For example,

`ball.py`

In Spyder, create a new file in the Editor, write or paste the Python code, save the file with the extension `.py`, and run it. The output of the program is normally displayed in the IPython Console.

Important: Spyder is not Python. Python is the programming language, while Spyder is only a development environment that makes writing, running, debugging and inspecting Python programs convenient.

Spyder is recommended for this course because its workflow is similar to MATLAB and it provides convenient tools for numerical computations. However, you are free to use any Python editor or development environment you prefer, such as Visual Studio Code (VS Code). You can also write Python code using a simple text editor and run the resulting `.py` file with a Python interpreter, for example from a Linux terminal. Python source code is plain text, so you can write a `.py` file using essentially any text editor or development environment and then execute it using a Python interpreter.

For example, on a Linux computer you can write a file called `ball.py` using any text editor and run it from a terminal with

```
python3 ball.py
```

Depending on the operating system and Python installation, the command may also be

```
python ball.py
```

The important point is that the Python code itself does not depend on Spyder.

Task 1: Python basics and error messages (1 point)

First, create a Python file called `ball.py` and copy the code below into the file.

Run the original program once and make sure that it works.

Then perform the modifications a)–f) **one at a time**. After each experiment, restore the original program before making the next modification.

For each case, write the error message in your report and briefly explain

- what caused the error, and
- how the error can be corrected.

- Insert the bare word `hello` on an empty line above the assignment to `v0`.
- Remove the leading `#` from the line `#Initial velocity (m/s)` so that the comment becomes ordinary program text.
- Remove the `=` sign in the assignment to `v0` so that the line becomes `v0 5.0`.
- In the expression `t**2`, replace `**` with `^` and observe what happens. Note that `^` does **not** mean exponentiation in Python.
- Add a bare `x` on a new line just before the line where `y` is calculated.
- Change `print (y)` to `print (z)` without defining the variable `z`.

Program for Task 1

```
if __name__ == "__main__":  
  
    # Program for computing the height of a ball  
    # in vertical motion  
  
    # Initial velocity (m/s)  
    v0 = 5.0  
  
    # Acceleration of gravity (m/s**2)  
    g = 9.81  
  
    # Time (s)  
    t = 0.6  
  
    # Vertical position (m)  
    y = v0*t - 0.5*g*t**2  
  
    print (y)
```

If you are using Spyder, you can create the file directly in the Editor and run it from there.

Task 2: Determinant and rank of a matrix (1 point)

Use NumPy to compute the determinant and rank of the matrix A presented in the code below.

Write the values of the determinant and rank in your report.

Based on the mathematics courses you have taken, briefly explain what the determinant and rank tell you about the matrix A . In particular, consider whether the matrix is singular or nonsingular and whether the corresponding system of equations has a unique solution.

```

import numpy as np

if __name__ == "__main__":

    # Create the vector x
    x = np.zeros((2, 1))
    x[0, 0] = 3.0
    x[1, 0] = 2.0

    # Create matrix A
    A = np.zeros((2, 2))
    A[0, 0] = 1.0
    A[0, 1] = 0.0
    A[1, 0] = 0.0
    A[1, 1] = 1.0

    # Matrix-vector product
    y = A @ x

    print("A =")
    print(A)

    print("x =")
    print(x)

    print("A @ x =")
    print(y)

    # Compute determinant and rank here

```

Hint: NumPy contains functions for both operations. You can inspect their documentation directly in Python by writing

```

help(np.linalg.det)
help(np.linalg.matrix_rank)

```

Task 3: Stiffness matrix of a two-node axial rod element (3 points)

Write a Python function that computes the local stiffness matrix of a two-node axial rod/bar element,

$$\overline{\mathbf{K}} = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \quad (3)$$

Use the parameter values

$$E = 1, \quad A = 1, \quad L = 1. \quad (4)$$

These values are not physically realistic. They are used only to keep the numerical values simple and to make the mathematical properties of the stiffness matrix easy to see.

Compute the

- determinant,

- rank, and
- eigenvalues

of the full element stiffness matrix.

Then apply a displacement boundary condition by fixing one nodal displacement. Do this by eliminating either

- the first row and first column, or
- the second row and second column

of the stiffness matrix.

Compute the determinant, rank and eigenvalues of the resulting reduced stiffness matrix.

In your report, explain the following:

- Why is the full element stiffness matrix singular before boundary conditions are applied?
- What is the physical meaning of the rank deficiency?
- What changes after one nodal displacement is fixed?
- Why can the reduced stiffness matrix be used to solve the unknown displacement?

Think also about what happens if both nodal displacement coordinates are given the same displacement. Does the element deform in this case?

Python function

Write a function for the element stiffness matrix.

```
import numpy as np

def Kloc(E, A, L):
    """
    Local stiffness matrix of a two-node axial rod element.

    Parameters
    -----
    E : Young's modulus
    A : cross-sectional area
    L : element length

    Returns
    -----
    K : 2 x 2 local element stiffness matrix
    """

    # Write the stiffness matrix here

    return K
```

Main program

Write a main program that calls the function and computes the requested quantities.

The following code gives you the basic structure of the program.

```
import numpy as np

if __name__ == "__main__":

    # Parameters
    E = 1.0
    A = 1.0
    L = 1.0

    # Element stiffness matrix
    K = Kloc(E, A, L)

    # Determinant, rank and eigenvalues
    # of the full stiffness matrix
    detK = np.linalg.det(K)
    rankK = np.linalg.matrix_rank(K)
    eigK = np.linalg.eigvalsh(K)

    print("Full stiffness matrix K:")
    print(K)

    print("det(K) =", detK)
    print("rank(K) =", rankK)
    print("eigenvalues(K) =", eigK)

    # Apply a displacement boundary condition.
    # Example: displacement of node 1 is fixed.
    #
    # Eliminate the first row and first column.
    Kred = K[1:, 1:]

    # Determinant, rank and eigenvalues
    # of the reduced stiffness matrix
    detKred = np.linalg.det(Kred)
    rankKred = np.linalg.matrix_rank(Kred)
    eigKred = np.linalg.eigvalsh(Kred)

    print("Reduced stiffness matrix Kred:")
    print(Kred)

    print("det(Kred) =", detKred)
    print("rank(Kred) =", rankKred)
    print("eigenvalues(Kred) =", eigKred)
```

Note that `np.linalg.eigvalsh` is used here because the stiffness matrix is symmetric.

Points

The maximum number of points is 5.

- Task 1: Correct error messages and explanations (1 point)

- Task 2: Correct determinant and rank of matrix A , with interpretation (1 point)
- Task 3: Correct full stiffness matrix and its determinant, rank and eigenvalues (1 point)
- Task 3: Correct reduced stiffness matrix and its determinant, rank and eigenvalues after applying the boundary condition (1 point)
- Task 3: Clear Python code and clear explanation of the physical meaning of the results (1 point)

Submission

Submit the following files in Moodle:

- A short report in PDF format
- The main Python file, for example `main_bonus1.py`
- The Python function file, for example `Kloc.py`

The report must include the requested numerical results, short explanations and the main conclusions.

The Python source files must be submitted as plain-text files with the `.py` extension.

Please note that points are given only if both the report (in PDF format) and the Python source codes (in `.py` format) are submitted.